



GraphGuard: The CHARM Critic

Catching Hallucinations by Reading the Attention Graph

CHARM stands for **Critical Hallucination Analysis via Residual Message-passing**—a framework that treats LLM generations as dynamic directed graphs.

- **The Thesis:** LLMs leave a "digital footprint" of internal reasoning through their attention mechanism
- **The Method:** We analyse *how the model looked at itself* while generating text, not just what it said
- **The Goal:** Predict a Hallucination Risk Score (0.0 to 1.0) by analysing graph topology

The Problem: The "Confidence" Illusion

Why Text-Only Detection Fails

Traditional hallucination detection methods fail because they analyse only the surface text, not the internal reasoning process that produced it.



01

Semantic Plausibility

Hallucinations often sound perfectly logical. Keyword matching or NLI can be fooled by a model that "speaks" with high linguistic fluency but zero factual grounding.

02

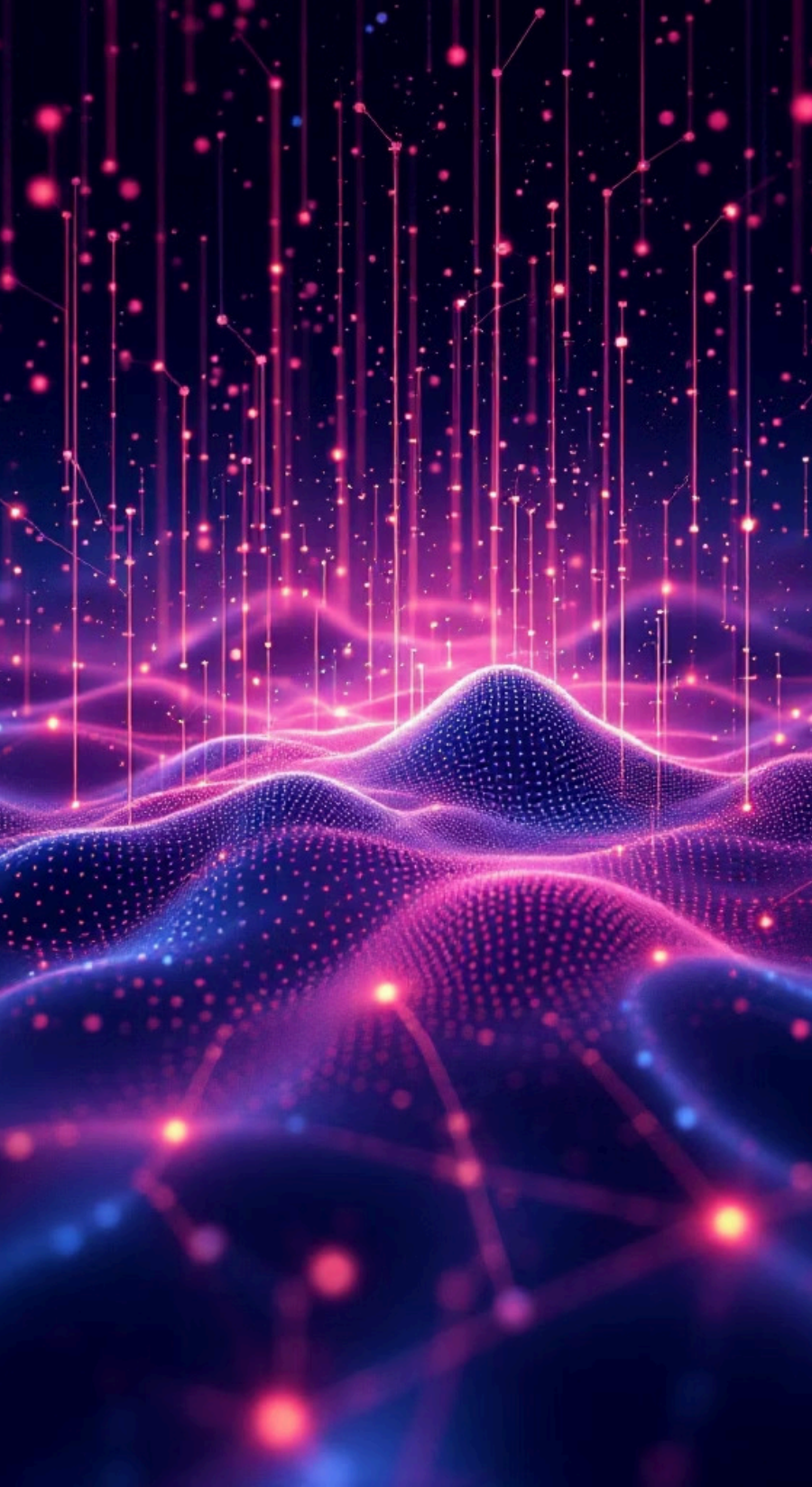
Calibrated Lying

Models can be trained to *look* confident using tags like , but this is often just another learned behaviour, not a reflection of internal state.

03

The Flattening Problem

Standard detectors treat output as a flat sequence of words. This destroys the **relational structure**—the specific logical dependencies between tokens.



The Key Insight: Attention as a Reasoning Chain

Grounding vs. Confabulation

When an LLM generates a correct, grounded answer, its internal **Attention Mechanism** behaves fundamentally differently than when it hallucinates.

Grounded Reasoning

Tight Back-References: Each new token strongly attends to specific prior tokens that provided logical foundation—attending back to "Context" or previous "Steps in a maths problem."

Hallucinated Reasoning

Diffuse or Self-Referential Attention: The model generates text that sounds right but lacks anchors. It attends weakly to context and mostly to its own immediately preceding "fluff," creating a circular reasoning loop.

The Pipeline: From Raw Data to Graph Scores

The process follows a strict 4-stage logical flow that transforms LLM generations into analysable graph structures.



The Fence

Force the LLM into structured output format using a **Grammar Fence** (DFA). This ensures we can always find the "Answer" and "Reasoning."



The Surgery

Use **Forward Hooks** to intercept the model's internal signals—Hidden States and Attention Weights—during generation.



The Packaging

Convert signals into a **PyTorch Geometric (PyG)** graph where Nodes = Generated Tokens and Edges = Strongest Attention Links.



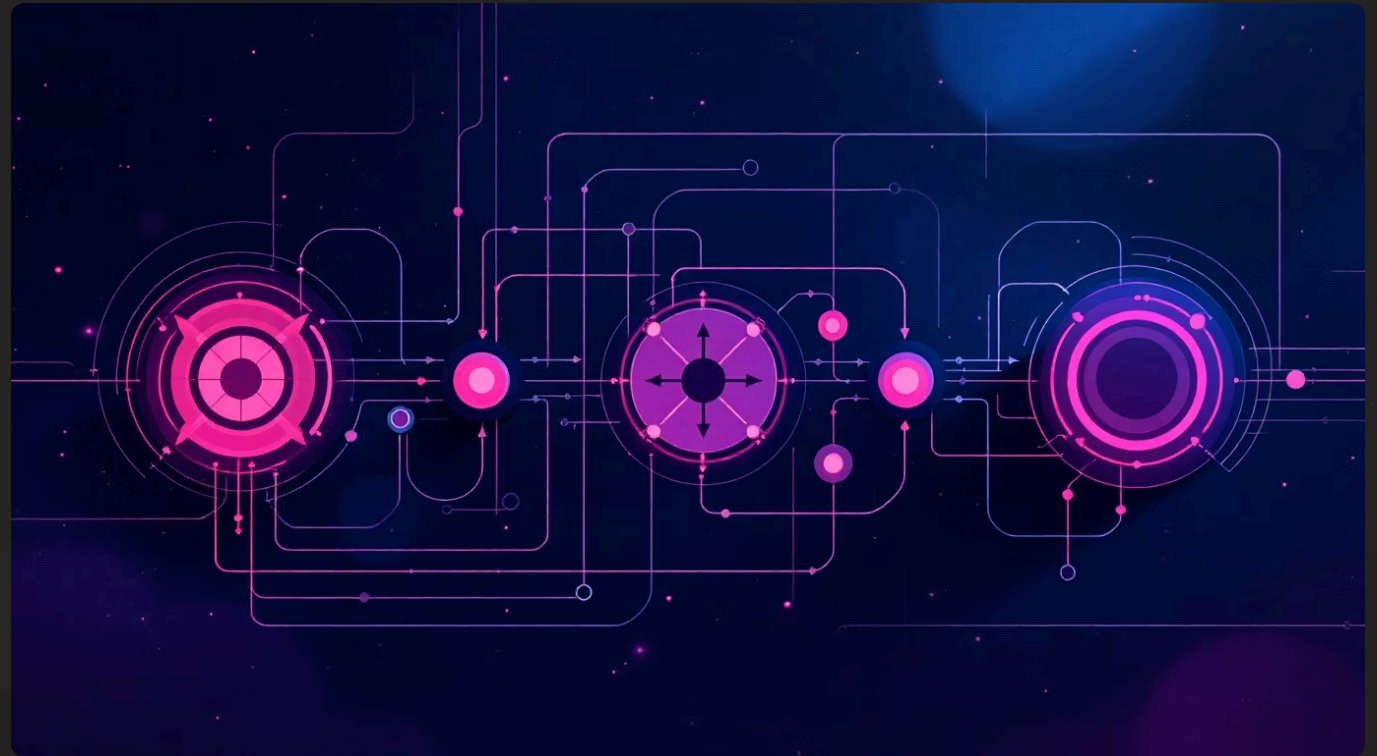
The Critic

Pass this graph through a **Graph Neural Network (GNN)** to classify the topology as "Faithful" or "Hallucinated."

The Grammar Fence

Hard-Coding the Format with Finite Automata

We use a **Deterministic Finite Automaton (DFA)** to mask the model's choices at every single step, ensuring mathematical guarantees rather than probabilistic suggestions.



The Constraint

The model *must* produce a block, an block, and a block in that exact order.

Why It Matters

Prompt engineering is a "suggestion." A Grammar Fence is a **mathematical guarantee** that cannot be violated.

The Benefit

Prevents model from "escaping" format mid-hallucination, making internal signals perfectly comparable across thousands of samples.

Internal Signal Extraction

Hidden States and Sparse Edges

We surgically extract two specific signals from the **final transformer layer** during generation.



The "Thought" (Hidden States)

A high-dimensional vector representing the model's semantic understanding of that specific token at the moment of generation.



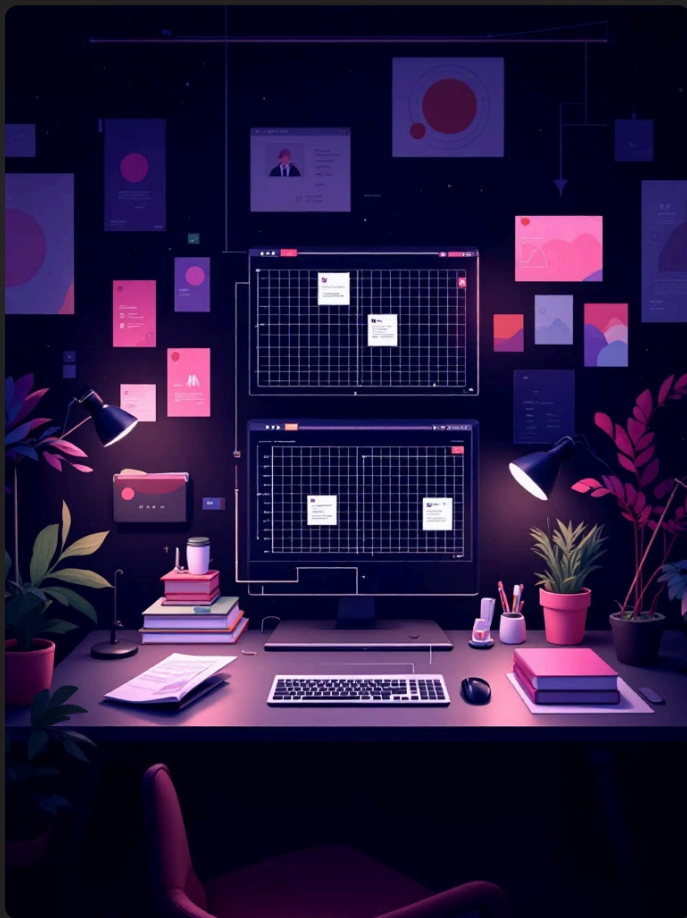
The "Link" (Attention Weights)

A distribution showing which prior tokens the current one is relying on. We ignore noise by only keeping "strong" links (>5% attention weight).

📄 **The Lookback Ratio:** We calculate how much the model looks at the provided **Context** versus its own **Generated Text**, revealing grounding vs. self-referential behaviour.

The Dataset & The Oracle

How the Critic Learns to Spot Lies



To train our GNN, we need "Ground Truth"—an Oracle that can label hallucinations with certainty.

Auto-Labeling



Run model on benchmarks (Math and Fact Retrieval). If doesn't match Gold Truth, label entire generation graph as **Hallucination** (Target: 1).

Inversion



Correct Answer → Faithful Graph Topology
Incorrect Answer → Hallucinated Graph Topology

The Result



A dataset of "Pathological Graphs" vs. "Healthy Graphs" for supervised learning.

The GNN Architecture (CHARMCritic)

Message Passing on Reasoning Chains

The GNN (Graph Attention Network) doesn't just look at tokens—it "walks" the graph, aggregating information across multiple hops to understand reasoning topology.



Layer 1: Direct Neighborhood

Each token aggregates information from tokens it directly attended to. It learns the *local* context of a claim.



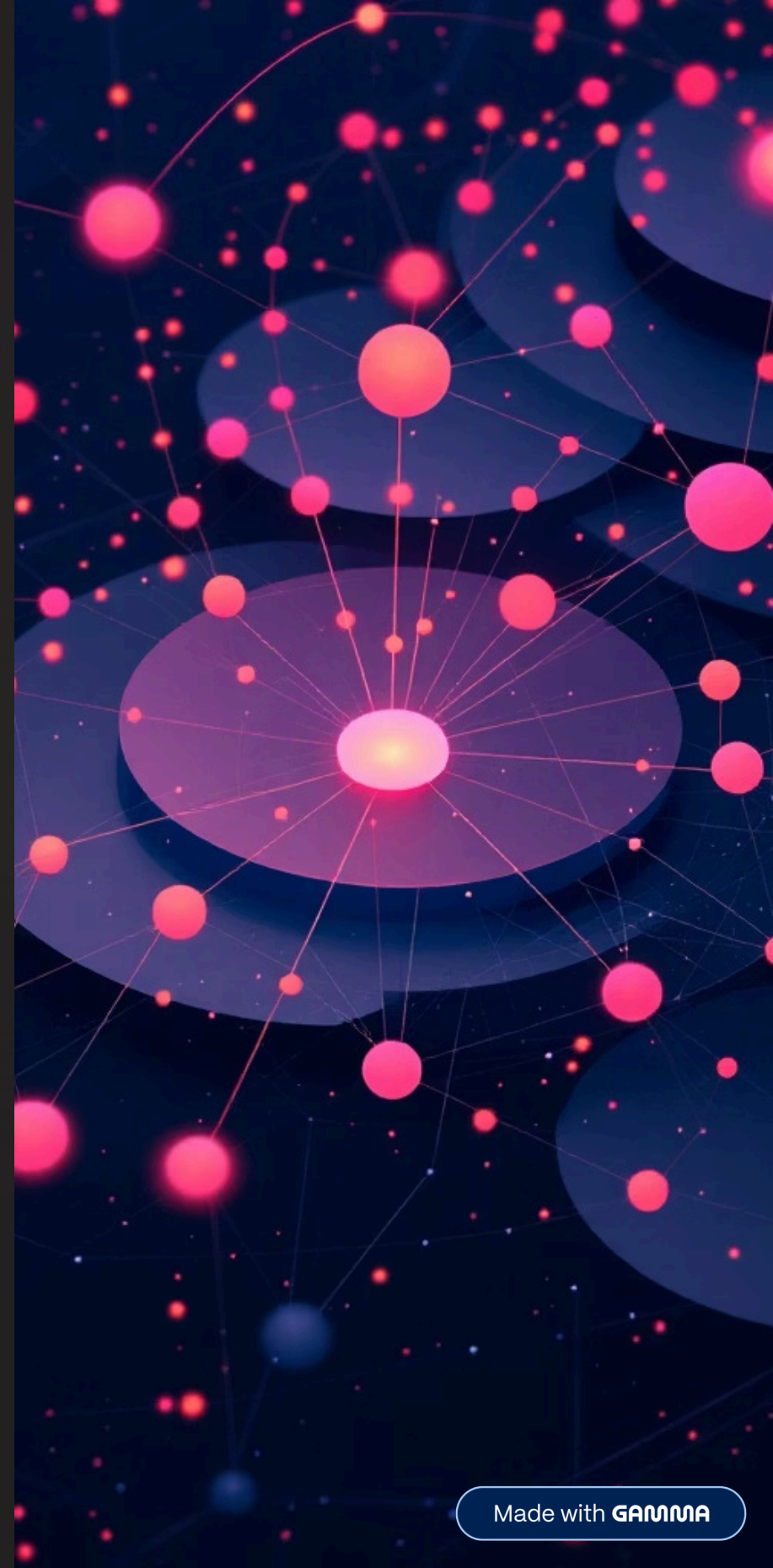
Layer 2: Multi-Hop Reasoning

Tokens aggregate information from "neighbours of neighbours." This allows detection of long-range reasoning chains or isolated "islands" of text.



Global Readout

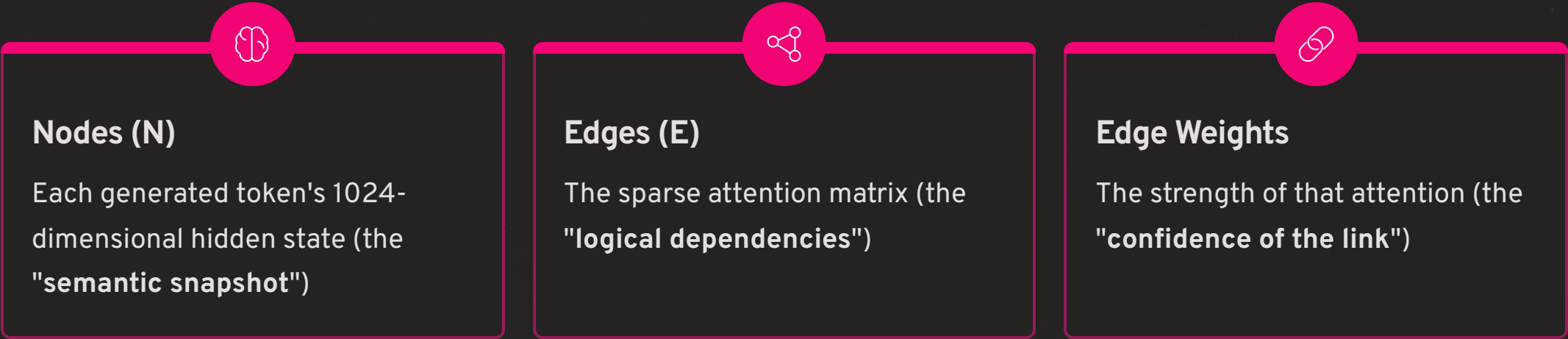
The entire graph collapses into a single vector. The GNN judges the **entire topology** to decide if information flow looks like factual retrieval or creative confabulation.



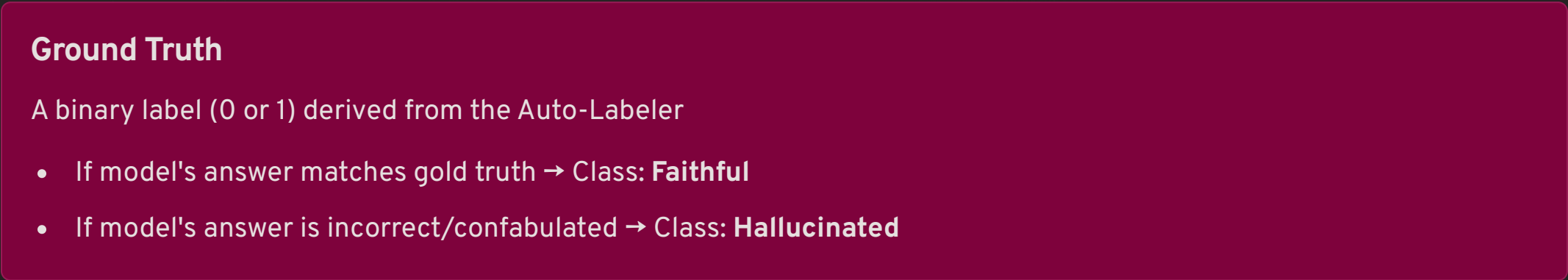
The Graph is the Feature, the Oracle is the Label

We aren't training the GNN on text strings; we are training it on the structural fingerprints of the model's internal state.

The Input (Features X, A)

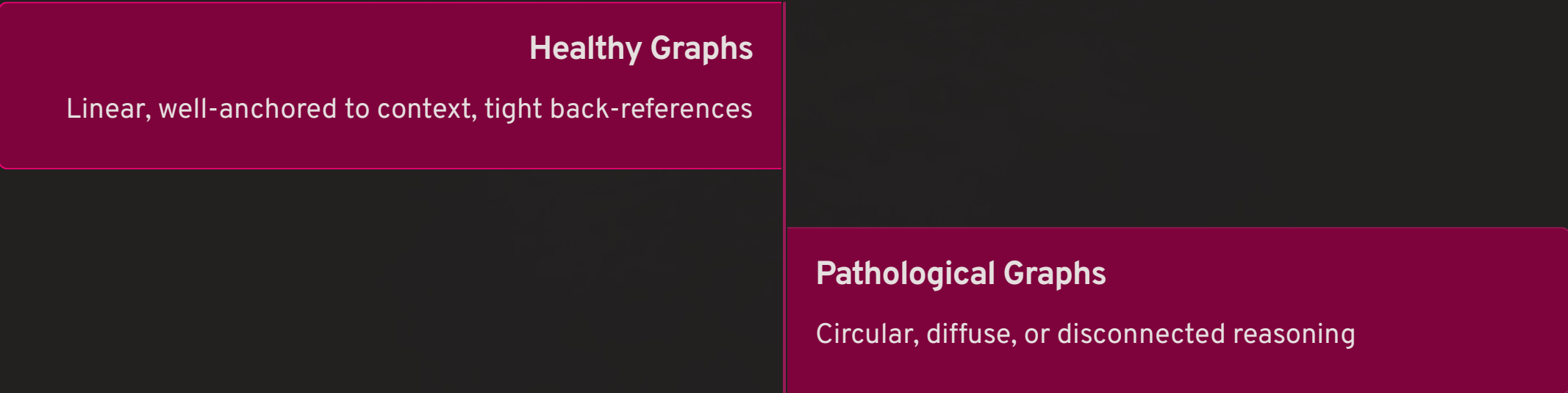


The Target (Label y)



The Learning Objective

The GNN learns to distinguish between two fundamental graph topologies:



**Key Takeaway:** The GNN becomes an expert at spotting "structural red flags" in the way an LLM attends to itself, allowing it to flag hallucinations even when the output text looks perfectly plausible.

Why Graph Structure Matters

GNN vs. Traditional "Bag of Tokens" Baselines

Feature	Baseline (Mean Pooling)	CHARM (GNN)
Input	Average of all token vectors	The full Attention Graph
Context	Sees <i>what</i> words were used	Sees <i>who</i> attended to <i>whom</i>
Logic	Ignored	Captured via edge topology
Hallucination Detection	Misses "fluent lies" (right words, wrong links)	Catches "fluent lies" via structural anomalies

❏ **The "Pathological" Signature:** Hallucinations often show "Attention Sinks" or "Disconnected Components" where the model stops looking at the facts and starts looking only at its own previous mistakes.

Summary & Interpretation

Scoring the "Trustworthiness" of a Generation

The final output is a score from **0.0 (Faithful)** to **1.0 (Hallucinated)**.

0.0 – 0.3

The model followed a clear, grounded reasoning path with tight back-references to context.

0.4 – 0.6

"Foggy" reasoning. The model is likely guessing or has weak grounding in the provided facts.

0.7 – 1.0

Structural collapse. The attention graph shows the model is "hallucinating into the void."

Key Takeaway: By shifting from **Natural Language Processing** to **Graph Analysis**, we treat hallucination as a structural defect in the reasoning chain, making detection much more robust than text-based methods.